

Faculty of Computers and Information Systems

Information Retrieval
Practical Laboratory

Week 1: Python File Manipulation for Information Retrieval

Create a small collection of text files and load it into a Python dictionary. This collection is the input to the retrieval programs developed in later weeks.

Duration: approximately 2 practical hours. Suggested pacing: 15 minutes concept review, 25 demonstration, 50 guided implementation, 20 selected exercises, and 10 review. Finish remaining exercises after the session.

1. Lab Objectives

- Create a docs directory containing three UTF-8 text files.
- Read, write, and append text using context managers.
- Load several files into a dictionary keyed by document ID.
- Compare `read()`, `readline()`, and `readlines()` using the same file.

2. Required Background

Prerequisite: basic Python variables, conditions, loops, lists, dictionaries, sets, and functions. No retrieval algorithms are required yet.

A document is one unit that a retrieval system can return, such as a text file. A collection is the set of documents being searched. In this lab, a filename without `.txt` becomes a document ID, and the dictionary stores each ID with its text.

A relative path is interpreted from the current working directory. The supplied program first finds the directory containing its own script, then joins `docs` or `data` to that directory. This lets the program run from a terminal opened elsewhere. `os.path.join` builds paths using the correct Windows separator.

Read mode `r` opens an existing file without changing it. Write mode `w` creates a file or replaces its contents. Append mode `a` adds text to the end. A `with` block closes the file even when an error occurs. Specify UTF-8 when creating and reading the collection.

`read()` returns the remaining text as one string. `readline()` reads one line. `readlines()` returns the remaining lines as a list. These operations advance the file position. Reopen the file to start reading from the beginning.

Course material: Preparation for the `os`-based document loaders in `section-2.py` and `5.py`.

3. Python Concepts Used

- **open and with:** Open a file and close it automatically after the indented block.
- **File modes:** Use `r` to read, `w` to write, and `a` to append.
- **os.path.exists:** Check whether a path already exists before creating a file.
- **os.listdir and sorted:** Get directory entries in a repeatable order.
- **os.path.join and os.path.isfile:** Build a path and exclude directories from file loading.
- **Dictionaries and loops:** Associate each document ID with its complete text.

4. Worked Example

Run this complete example inside a disposable practice directory. The first block creates a file; the second reads it. The shorter form requested in the course material works with the platform default encoding; the full laboratory program adds `encoding="utf-8"` for predictable text handling.

```
with open("doc1.txt", "w") as f:
    f.write("Python helps us process text.\n")
with open("doc1.txt", "r") as f:
    text = f.read()
print(text.strip())
```

The output is Python helps us process text. The variable text remains available after the file is closed. In the full program, a dictionary accumulates three such strings. The demonstration notes file deliberately uses w before a, so rerunning it gives the same two lines.

5. Step-by-Step Practical Task

Step 1: Open the Week_01_File_Manipulation folder in your editor. Identify data/welcome.txt and the two Python files.

Step 2: Predict what each file-reading method will return. Run the solution once to create docs and the demonstration notes file.

Step 3: Inspect docs/doc1.txt, doc2.txt, and doc3.txt. Record each document ID and its text.

Step 4: Trace the sorted directory loop. Explain why both the .txt suffix and isfile check are used.

Step 5: Modify a document, rerun the solution, and confirm that your edit is preserved.

Step 6: Complete the starter loader and print every dictionary key and value. Run it from a different working directory.

Execution: open a terminal in this week folder and run python lab01_file_manipulation.py. On Windows, py -3 can replace python. All required files are included; no internet or third-party Python packages are needed.

6. Complete Python Program

The listing below matches lab01_file_manipulation.py exactly. Keep indentation when copying. The exercise starter is separate and intentionally unfinished.

lab01_file_manipulation.py

```
"""Week 1: create and load a small UTF-8 document collection."""
import os

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

def load_documents(folder_path):
    documents = {}
    for filename in sorted(os.listdir(folder_path)):
        path = os.path.join(folder_path, filename)
        if filename.endswith(".txt") and os.path.isfile(path):
            with open(path, "r", encoding="utf-8") as file:
                documents[filename[:-4]] = file.read()
    return documents

def main():
    folder = os.path.join(BASE_DIR, "docs")
    if not os.path.exists(folder):
        os.makedirs(folder)
    samples = {
        "doc1": "Python supports data analysis.\n",
        "doc2": "Java supports web development.\n",
        "doc3": "Networks connect computers.\n",
    }
    for name, text in samples.items():
        path = os.path.join(folder, name + ".txt")
        if not os.path.exists(path): # Preserve student edits on reruns.
            with open(path, "w", encoding="utf-8") as file:
                file.write(text)
```

```
# This disposable demonstration file is reset on each run.
notes_path = os.path.join(BASE_DIR, "data", "notes.txt")
with open(notes_path, "w", encoding="utf-8") as file:
    file.write("First line\n")
with open(notes_path, "a", encoding="utf-8") as file:
    file.write("Appended line\n")
with open(notes_path, "r", encoding="utf-8") as file:
    print("readline:", repr(file.readline()))
    print("remaining readlines:", file.readlines())
with open(notes_path, "r", encoding="utf-8") as file:
    print("read:", repr(file.read()))
for name, text in load_documents(folder).items():
    print(name, "->", text.strip())

if __name__ == "__main__":
    main()
```

7. Expected Output

Representative output from the shipped collection and the default command. Calculated values come from a tested program run. Long output lines wrap here for printing; expected_output.txt contains the full terminal output. Selected rows are shown for large matrices.

```
readline: 'First line\n'
remaining readlines: ['Appended line\n']
read: 'First line\nAppended line\n'
doc1 -> Python supports data analysis.
doc2 -> Java supports web development.
doc3 -> Networks connect computers.
```

The transcript assumes the newly created three-document collection. Existing student edits are preserved, so later runs may print different document contents.

8. Laboratory Exercises

1. Basic: add doc4.txt about databases and confirm that the loader finds it.
2. Implementation: finish load_documents in exercises_lab01.py using with open and UTF-8.
3. Investigation: call readline() twice and then read(). Predict the three results before running.
4. Debugging: explain why opening an existing document in w mode before reading loses its earlier text. Reproduce this only in a disposable file.
5. Interpretation: place a subdirectory and a .csv file in docs. Explain why neither should become a document.
6. Extension: print the number of characters and lines for each loaded document.

9. Student Tasks

- Your completed exercise file and a docs folder with at least four short documents.
- A terminal transcript showing all document IDs and contents, plus one run from another directory.
- A short explanation of the difference between the three read methods and the effects of w and a.

Submit your own implementation and evidence. Do not submit an untouched starter as a completed solution. Label all outputs with the query or input that produced them.

10. Review Questions

1. What is the difference between a document and a collection?
2. Which directory supplies the starting point for an ordinary relative path?

3. Why is a with block useful for file operations?
4. Why do successive read operations return different parts of a file?
5. Why should the document loader skip subdirectories?

11. Challenge Exercise

Optional: Write a separate collection summary file containing one line per document: ID, character count, and line count. Keep it outside docs so it does not become a searchable document.